

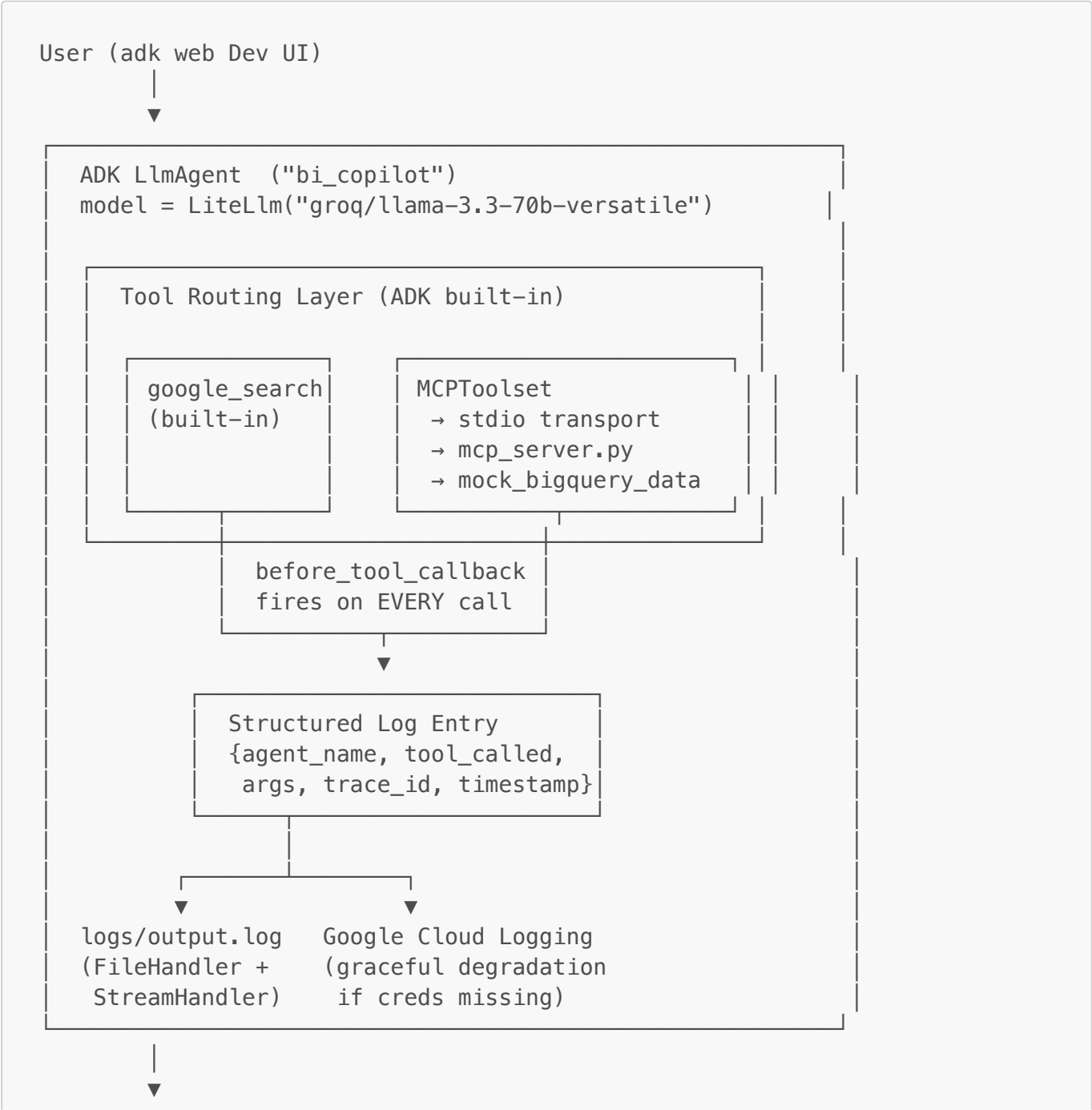
# ADK Multi-Agent BI Co-Pilot (Doc Governance Pattern)

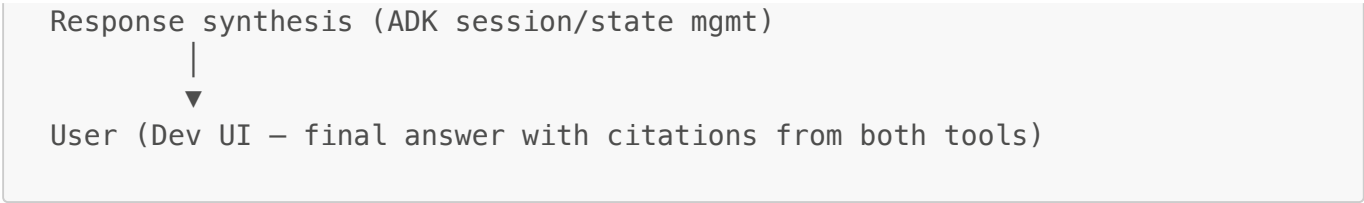
## — Complete Guide

### 1. Project Architecture & Overview

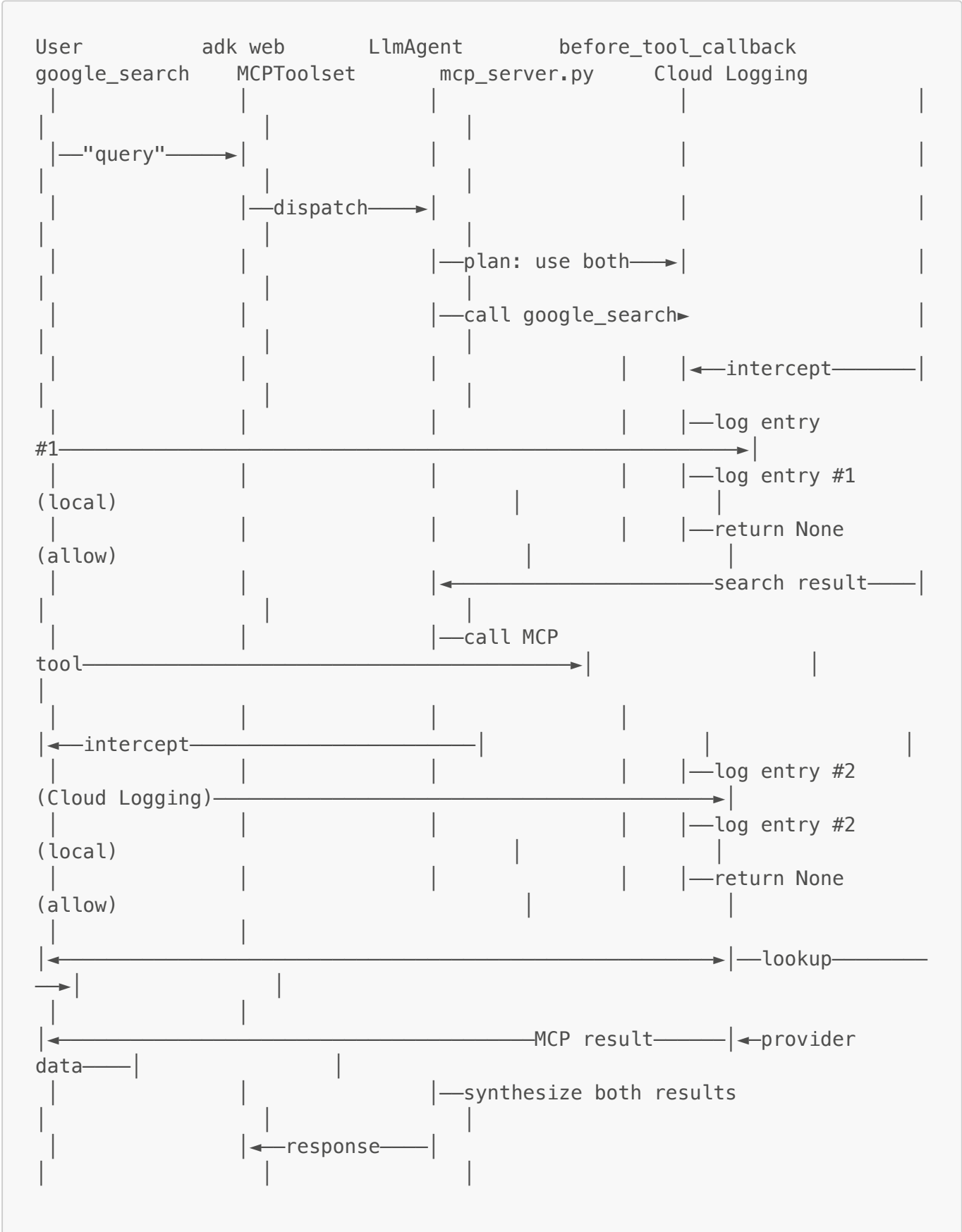
#### End-to-End Pipeline Topology

This project builds a **Google Agent Development Kit (ADK) LlmAgent** that acts as a Business Intelligence Co-Pilot. The agent can simultaneously search the public web via the built-in `google_search` tool **and** query an internal mock data-availability registry via a custom MCP (Model Context Protocol) server. Every tool call — regardless of source — is intercepted by a `before_tool_callback` that emits a structured, dual-sink log entry (local file + Google Cloud Logging).





Sequence Diagram — Single Query Touching Both Tool Sources





The callback fires **twice** for one user turn — once when `google_search` is invoked, once when `check_provider_data_availability` is invoked. Each firing produces an independent `trace_id` and log entry.

## Technology Justifications

**Why Google ADK over raw LangChain / manual orchestration?** ADK provides built-in session and state management across turns, a ready-made Dev UI (`adk web`) for rapid iteration, a native evaluation harness, and first-class `MCPToolset` integration — all without hand-rolling context accumulation, tool-schema injection, or server-sent-event streaming. A raw LangChain loop would require implementing all of that plumbing manually and would not produce the standardised session artifacts ADK emits.

**Why MCP for the custom tool?** MCP (Model Context Protocol) decouples agent logic from data-access logic via a standardised JSON-RPC-over-stdio (or SSE) interface. This means the mock BigQuery server (`mcp_server.py`) can be swapped for a real BigQuery-backed MCP server — or any other data source — **without touching `agent.py`**. It also means the tool schema is discovered at runtime rather than hard-coded, enabling zero-change extensibility as new tools are added to the server.

**Why LiteLLM bridge for Groq?** ADK's `LlmAgent` natively expects a Gemini model string. `LiteLlm` is ADK's official escape hatch that translates any OpenAI-compatible provider (Groq, Anthropic, Azure, etc.) into the interface ADK expects, keeping all of ADK's native abstractions (tool injection, streaming, session management) intact. Groq is chosen for its low-latency inference on `llama-3.3-70b-versatile` at a fraction of the cost of hosted frontier models.

## Why structured dual-sink logging?

- **Local `output.log`**: enables fast debugging during development without any cloud dependency or network round-trip. The file is always written even if the agent is running offline.
- **Google Cloud Logging**: enables centralised, queryable, team-visible observability in production — including log-based metrics, alerting, and audit trails across multiple agent instances. The graceful-degradation try/except wrapper ensures a missing `GOOGLE_CLOUD_PROJECT` or missing ADC credentials does not block the agent in a dev/CI environment.

## 2. Repository & Folder Structure

### Full ASCII Tree

```

adk_agent/
├── agent.py           # LlmAgent definition – ADK entry point
├── callbacks.py      # before_tool_callback with dual-sink
├── logging
│   ├── mcp_server.py # Mock Day-8 MCP server (stand-in)
│   ├── test_queries.md # 5 example dual-tool queries
│   └── test_results.md # Observed results from running those
└── queries

```

```

├─ cloud_logging_screenshot.png    # Runtime artifact (intern produces at
runtime)
├─ requirements.txt                # Pinned Python dependencies
├─ setup.sh                        # One-shot scaffold + venv creation
script
├─ README.md                      # Empty – intern fills in
├─ .env.example                   # Template for required env vars
├─ .gitignore                     # Standard Python + secrets gitignore
├─ config/
│   └─ settings.py                # Env-var loader and validator
├─ utils/
│   └─ logger_config.py           # Custom dual-sink logger factory
│   └─ mock_bigquery_data.py      # In-memory provider dataset + lookup
fn
├─ tests/
│   └─ test_agent_tools.py        # pytest unit tests
└─ logs/
    └─ output.log                 # Generated at runtime (not authored)

```

## setup.sh — Complete One-Shot Scaffold Script

```

#!/usr/bin/env bash
# setup.sh – Scaffolds the entire adk_agent/ project in one shot.
# Run from the PARENT directory of adk_agent/, e.g.:
#   chmod +x setup.sh && ./setup.sh

set -euo pipefail

echo "==> Creating directory structure..."
mkdir -p adk_agent/config
mkdir -p adk_agent/utils
mkdir -p adk_agent/tests
mkdir -p adk_agent/logs

echo "==> Creating placeholder files..."
touch adk_agent/agent.py
touch adk_agent/callbacks.py
touch adk_agent/mcp_server.py
touch adk_agent/test_queries.md
touch adk_agent/test_results.md
touch adk_agent/requirements.txt
touch adk_agent/.env.example
touch adk_agent/.gitignore
touch adk_agent/config/settings.py
touch adk_agent/config/__init__.py
touch adk_agent/utils/logger_config.py
touch adk_agent/utils/mock_bigquery_data.py
touch adk_agent/utils/__init__.py
touch adk_agent/tests/test_agent_tools.py
touch adk_agent/tests/__init__.py
touch adk_agent/logs/output.log

```

```
touch adk_agent/__init__.py

# README is intentionally left empty for the intern to fill in.
touch adk_agent/README.md

echo "==> Creating Python virtual environment (myenv)..."
python3 -m venv adk_agent/myenv

echo ""
echo "==> Scaffold complete. Next steps:"
echo "    1. cd adk_agent"
echo "    2. source myenv/bin/activate"
echo "    3. Paste all source files from guide.md into their respective
paths."
echo "    4. cp .env.example .env && edit .env with your real keys."
echo "    5. pip install -r requirements.txt"
echo "    6. python mcp_server.py    # sanity-check the MCP server
standalone"
echo "    7. adk web                # launch the ADK Dev UI"
```

### 3. Production-Ready Implementation Code

#### config/settings.py

```
# config/settings.py
"""
Loads and validates all required environment variables for the ADK BI Co-
Pilot.
Uses python-dotenv to read from a .env file if present, with os.environ as
fallback.
Raises a clear RuntimeError on startup if any required key is absent.
"""

import os
from pathlib import Path
from dotenv import load_dotenv

# Resolve .env relative to this file's parent (adk_agent/)
_ENV_PATH = Path(__file__).resolve().parent.parent / ".env"
load_dotenv(dotenv_path=_ENV_PATH, override=False)

def _require(key: str) -> str:
    """Return the value of an env var, raising RuntimeError if it is
missing or empty."""
    value = os.environ.get(key, "").strip()
    if not value:
        raise RuntimeError(
            f"[settings] Required environment variable '{key}' is not set.
"

```

```

        f"Add it to your .env file or export it in your shell before
        running."
    )
    return value

def _optional(key: str, default: str = "") -> str:
    """Return the value of an optional env var, or a default if absent."""
    return os.environ.get(key, default).strip()

# -----
-- #
# Required
# -----
-- #
GROQ_API_KEY: str = _require("GROQ_API_KEY")

# -----
-- #
# Optional – graceful degradation if absent (Cloud Logging will be
# skipped)
# -----
-- #
GOOGLE_CLOUD_PROJECT: str = _optional("GOOGLE_CLOUD_PROJECT")
GOOGLE_APPLICATION_CREDENTIALS: str =
_optional("GOOGLE_APPLICATION_CREDENTIALS")

# -----
-- #
# Derived / static
# -----
-- #
AGENT_NAME: str = "bi_copilot"
GROQ_MODEL: str = "groq/llama-3.3-70b-versatile"
MCP_SERVER_SCRIPT: str = str(Path(__file__).resolve().parent.parent /
"mcp_server.py")
LOG_FILE_PATH: str = str(Path(__file__).resolve().parent.parent / "logs" /
"output.log")

```

---

## utils/logger\_config.py

```

# utils/logger_config.py
"""
Custom dual-sink logger factory.

Every logger produced by get_logger() writes simultaneously to:
1. stdout (StreamHandler) – for live terminal feedback
2. logs/output.log (FileHandler, append mode) – for persistent audit
trail

```

Log line format (exact, non-negotiable):

TIMESTAMP | LEVEL | MESSAGE

Example:

```
2025-09-12T14:03:22.417Z | INFO | Tool called: google_search |
trace_id=abc-123
"""
```

```
import logging
```

```
import sys
```

```
from pathlib import Path
```

```
# Ensure the logs/ directory exists before any FileHandler tries to open
it.
```

```
_LOG_DIR = Path(__file__).resolve().parent.parent / "logs"
```

```
_LOG_DIR.mkdir(parents=True, exist_ok=True)
```

```
_LOG_FILE = _LOG_DIR / "output.log"
```

```
_LOGGERS: dict[str, logging.Logger] = {}
```

```
class _PipeFormatter(logging.Formatter):
```

```
    """
```

```
    Produces lines in the exact format:
```

```
    TIMESTAMP | LEVEL | MESSAGE
```

```
    where TIMESTAMP is ISO 8601 UTC (millisecond precision).
```

```
    """
```

```
    def formatTime(self, record: logging.LogRecord, datefmt: str | None =
None) -> str: # noqa: N802
```

```
        import datetime
```

```
        dt = datetime.datetime.fromtimestamp(record.created,
```

```
        tz=datetime.timezone.utc)
```

```
        return dt.strftime("%Y-%m-%dT%H:%M:%S.") + f"{dt.microsecond //
1000:03d}Z"
```

```
    def format(self, record: logging.LogRecord) -> str:
```

```
        timestamp = self.formatTime(record)
```

```
        level = record.levelname
```

```
        message = record.getMessage()
```

```
        # Include exception info if present
```

```
        if record.exc_info:
```

```
            exc_text = self.formatException(record.exc_info)
```

```
            message = f"{message}\n{exc_text}"
```

```
        return f"{timestamp} | {level} | {message}"
```

```
def get_logger(name: str) -> logging.Logger:
```

```
    """
```

Return a named logger configured with both a StreamHandler and a FileHandler.

Calling get\_logger() multiple times with the same name returns the same instance

```

(idempotent) – handlers are never duplicated.
"""
if name in _LOGGERS:
    return _LOGGERS[name]

logger = logging.getLogger(name)
logger.setLevel(logging.DEBUG)
# Prevent log records from propagating to the root logger (avoids
duplicate output
# when the root logger also has handlers, e.g. during pytest).
logger.propagate = False

if not logger.handlers:
    formatter = _PipeFormatter()

    # 1. Terminal (stdout)
    stream_handler = logging.StreamHandler(sys.stdout)
    stream_handler.setLevel(logging.DEBUG)
    stream_handler.setFormatter(formatter)

    # 2. File (append mode)
    file_handler = logging.FileHandler(str(_LOG_FILE), mode="a",
encoding="utf-8")
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(formatter)

    logger.addHandler(stream_handler)
    logger.addHandler(file_handler)

_LOGGERS[name] = logger
return logger

```

## utils/mock\_bigquery\_data.py

```

# utils/mock_bigquery_data.py
"""
Simulates an internal BigQuery-style dataset of cloud provider usage data.

In the real Day-8 project this module would be replaced by actual BigQuery
client calls. Here we use an in-memory Python dict as a stand-in so the MCP
server can operate fully offline.

Schema per provider record:
{
    "provider_name":      str    – canonical name (case-insensitive lookup
key)
    "display_name":       str    – human-readable label
    "data_available":     bool   – True if internal usage data exists
    "record_count":       int    – approximate row count (0 if
data_available=False)

```



```

        "last_updated":      str    - ISO 8601 date of last data refresh
        "datasets":         list[str] - names of available internal datasets
        "notes":            str    - any caveats
    }
"""

from __future__ import annotations

_PROVIDER_DATASET: list[dict] = [
    {
        "provider_name": "aws",
        "display_name": "Amazon Web Services (AWS)",
        "data_available": True,
        "record_count": 1_847_302,
        "last_updated": "2025-09-01",
        "datasets": ["aws_cost_usage_v2", "aws_resource_inventory",
"aws_iam_audit"],
        "notes": "Full CUR export integrated. Data refreshed nightly.",
    },
    {
        "provider_name": "microsoft azure",
        "display_name": "Microsoft Azure",
        "data_available": True,
        "record_count": 924_611,
        "last_updated": "2025-09-01",
        "datasets": ["azure_cost_management", "azure_advisor_recalls",
"azure_policy_compliance"],
        "notes": "Integrated via Azure Cost Management API. Some
subscription gaps exist.",
    },
    {
        "provider_name": "google cloud",
        "display_name": "Google Cloud Platform (GCP)",
        "data_available": True,
        "record_count": 2_103_887,
        "last_updated": "2025-09-02",
        "datasets": ["gcp_billing_export", "gcp_asset_inventory",
"gcp_iam_recommender"],
        "notes": "Native BigQuery billing export. Most complete dataset.",
    },
    {
        "provider_name": "oracle cloud",
        "display_name": "Oracle Cloud Infrastructure (OCI)",
        "data_available": False,
        "record_count": 0,
        "last_updated": "N/A",
        "datasets": [],
        "notes": "OCI integration pending – contract under review as of Q3
2025.",
    },
    {
        "provider_name": "alibaba cloud",
        "display_name": "Alibaba Cloud",
        "data_available": True,

```

```

        "record_count": 312_045,
        "last_updated": "2025-08-15",
        "datasets": ["alibaba_billing_summary", "alibaba_ecs_usage"],
        "notes": "Partial integration. Billing granularity limited to daily
aggregates.",
    },
    {
        "provider_name": "ibm cloud",
        "display_name": "IBM Cloud",
        "data_available": False,
        "record_count": 0,
        "last_updated": "N/A",
        "datasets": [],
        "notes": "IBM Cloud integration was deprecated in 2024. No current
data pipeline.",
    },
]

```

```

# Build a lookup index keyed by normalised provider_name for O(1) access.
_INDEX: dict[str, dict] = {row["provider_name"].lower(): row for row in
_PROVIDER_DATASET}

```

```

# Also add common aliases so the LLM can use natural language names.

```

```

_ALIASES: dict[str, str] = {
    "amazon": "aws",
    "amazon web services": "aws",
    "azure": "microsoft azure",
    "gcp": "google cloud",
    "google cloud platform": "google cloud",
    "oci": "oracle cloud",
    "oracle": "oracle cloud",
    "alibaba": "alibaba cloud",
    "ibm": "ibm cloud",
}

```

```

def lookup_provider(provider_name: str) -> dict:

```

```

    """

```

```

    Look up a cloud provider by name (case-insensitive, alias-aware).

```

```

    Returns a dict with keys:

```

```

        found (bool), provider_name (str), display_name (str),
        data_available (bool), record_count (int), last_updated (str),
        datasets (list[str]), notes (str)

```

```

    If the provider is not recognised, returns found=False and an empty
record.

```

```

    """

```

```

    key = provider_name.strip().lower()
    # Resolve alias first
    canonical = _ALIASES.get(key, key)
    record = _INDEX.get(canonical)

```

```

    if record is None:

```

```

        return {
            "found": False,
            "provider_name": provider_name,
            "display_name": provider_name,
            "data_available": False,
            "record_count": 0,
            "last_updated": "N/A",
            "datasets": [],
            "notes": f"Provider '{provider_name}' is not in the internal
registry.",
        }

    return {
        "found": True,
        **record,
    }

def list_all_providers() -> list[str]:
    """Return the canonical names of all providers in the registry."""
    return [row["provider_name"] for row in _PROVIDER_DATASET]

```

---

## mcp\_server.py

```

# mcp_server.py
"""
Mock MCP Server – Day-8 Stand-In
=====
THIS FILE IS A STAND-IN FOR THE INTERN'S REAL DAY-8 MCP SERVER.
Replace the tool implementations below with calls to your actual Day-8
data sources (BigQuery, internal APIs, etc.) when moving to production.

This server exposes the following tool to any MCP-compatible client (e.g.,
ADK's MCPToolset):

    check_provider_data_availability(provider_name: str) -> dict
        Checks whether internal cloud-usage data exists for a given provider.
        Backed by the in-memory mock_bigquery_data dataset.

Transport: stdio (standard input/output)
Run standalone for a sanity check:
    python mcp_server.py
Then send a JSON-RPC call via stdin, or just let ADK's MCPToolset manage
it.
"""

import asyncio
import json
import sys
from pathlib import Path

```

```

# Ensure the project root is on the Python path so utils/ is importable
# regardless of the working directory from which the server is launched.
_PROJECT_ROOT = Path(__file__).resolve().parent
if str(_PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(_PROJECT_ROOT))

from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp import types as mcp_types

from utils.mock_bigquery_data import lookup_provider, list_all_providers
from utils.logger_config import get_logger

logger = get_logger("mcp_server")

# -----
-- #
# Server initialisation
# -----
-- #
app = Server("bi-copilot-mock-day8-mcp-server")

# -----
-- #
# Tool: list_tools
# -----
-- #
@app.list_tools()
async def list_tools() -> list[mcp_types.Tool]:
    """Advertise available tools to any connecting MCP client."""
    return [
        mcp_types.Tool(
            name="check_provider_data_availability",
            description=(
                "Check whether internal cloud-usage data is available for a
given "
                "cloud provider. Returns availability status, approximate
record count, "
                "last-updated date, and the names of available internal
datasets. "
                "Supported providers include AWS, Microsoft Azure, Google
Cloud, "
                "Oracle Cloud, Alibaba Cloud, and IBM Cloud."
            ),
            inputSchema={
                "type": "object",
                "properties": {
                    "provider_name": {
                        "type": "string",
                        "description": (
                            "The name of the cloud provider to look up,
e.g. "

```

```

        "'AWS', 'Microsoft Azure', 'Google Cloud'. "
        "Common aliases and abbreviations are
supported."

        ),
    },
    },
    "required": ["provider_name"],
},
),
mcp_types.Tool(
    name="list_all_providers",
    description=(
        "Return a list of all cloud providers that have records in
the "
        "internal data registry. Useful for enumerating what the
agent knows about."
    ),
    inputSchema={
        "type": "object",
        "properties": {},
        "required": [],
    },
),
]

```

```

# -----
-- #
# Tool: call_tool (dispatcher)
# -----
-- #
@app.call_tool()
async def call_tool(
    name: str,
    arguments: dict,
) -> list[mcp_types.TextContent]:
    """Dispatch incoming tool calls to the appropriate handler."""

    logger.info(f"MCP tool dispatched | tool={name} | args=
{json.dumps(arguments)}")

    if name == "check_provider_data_availability":
        provider_name = arguments.get("provider_name", "").strip()
        if not provider_name:
            result = {
                "error": "provider_name argument is required and must not
be empty.",
                "found": False,
            }
        else:
            # --- STAND-IN: Replace lookup_provider() with a real BigQuery
call here ---
            result = lookup_provider(provider_name)

```

```

        logger.info(
            f"check_provider_data_availability result | provider={
{provider_name}} "
            f"| found={result.get('found')} "
            f"| data_available={result.get('data_available')}}"
        )
        return [mcp_types.TextContent(type="text", text=json.dumps(result,
indent=2))]

    elif name == "list_all_providers":
        providers = list_all_providers()
        result = {"providers": providers}
        logger.info(f"list_all_providers result | count={len(providers)}")
        return [mcp_types.TextContent(type="text", text=json.dumps(result,
indent=2))]

    else:
        error_payload = {"error": f"Unknown tool: '{name}'. No handler
registered."}
        logger.warning(f"Unknown tool called: {name}")
        return [mcp_types.TextContent(type="text",
text=json.dumps(error_payload))]

# -----
-- #
# Entry point
# -----
-- #
async def main() -> None:
    logger.info("Mock Day-8 MCP server starting (stdio transport)...")
    async with stdio_server() as (read_stream, write_stream):
        await app.run(
            read_stream,
            write_stream,
            app.create_initialization_options(),
        )

if __name__ == "__main__":
    asyncio.run(main())

```

## callbacks.py

```

# callbacks.py
"""
ADK before_tool_callback
=====
Fires on EVERY tool invocation (both google_search and MCP tools).
Responsibilities:

```

1. Generate a unique trace\_id (UUID4) per invocation.
2. Build a structured log entry dict.
3. Write it to the local dual-sink logger (terminal + output.log).
4. Attempt a push to Google Cloud Logging; degrade gracefully on failure.
5. Return None so ADK continues to execute the tool unchanged.  
(Returning a non-None value would short-circuit the tool call – we intentionally do NOT do that here; this callback is observe-only.)

Lenient error-handling policy:

If either google\_search or the MCP tool raises an exception, the exception is caught in the agent's tool-runner (see agent.py). The callback itself never throws – any internal error is logged as ERROR and swallowed so the agent turn is never aborted by a logging failure.

"""

```
from __future__ import annotations
```

```
import json
```

```
import traceback
```

```
import uuid
```

```
from datetime import datetime, timezone
```

```
from typing import Any
```

```
from google.adk.tools import BaseTool
```

```
from google.adk.tools.tool_context import ToolContext
```

```
from config.settings import AGENT_NAME, GOOGLE_CLOUD_PROJECT
```

```
from utils.logger_config import get_logger
```

```
logger = get_logger("callbacks")
```

```
def _build_log_entry(
```

```
    tool_name: str,
```

```
    args: dict[str, Any],
```

```
    trace_id: str,
```

```
    timestamp: str,
```

```
) -> dict[str, Any]:
```

```
    """Construct the canonical structured log entry dict."""
```

```
    return {
```

```
        "agent_name": AGENT_NAME,
```

```
        "tool_called": tool_name,
```

```
        "args": args,
```

```
        "trace_id": trace_id,
```

```
        "timestamp": timestamp,
```

```
    }
```

```
def _push_to_cloud_logging(entry: dict[str, Any]) -> None:
```

```
    """
```

```
    Attempt to write a structured entry to Google Cloud Logging.
```

```
    Raises nothing – all exceptions are caught by the caller.
```

```
    """
```

```

import google.cloud.logging # type: ignore[import]
from google.cloud.logging import Client # type: ignore[import]

client = Client(project=GOOGLE_CLOUD_PROJECT if GOOGLE_CLOUD_PROJECT
else None)
cloud_logger = client.logger(f"adk-{AGENT_NAME}-tool-calls")
cloud_logger.log_struct(entry)

def before_tool_callback(
    tool: BaseTool,
    args: dict[str, Any],
    tool_context: ToolContext,
) -> None:
    """
    ADK before_tool_callback hook.

    Parameters
    -----
    tool          : The ADK BaseTool instance about to be called.
    args          : The argument dict the LLM generated for this tool call.
    tool_context  : ADK ToolContext carrying session state, agent metadata,
    etc.

    Returns
    -----
    None – returning None allows ADK to proceed with the actual tool
    execution.
    Returning a dict here would short-circuit the tool call; we
    deliberately
    avoid that so this callback is purely observational.
    """
    try:
        trace_id = str(uuid.uuid4())
        timestamp = datetime.now(tz=timezone.utc).isoformat()
        tool_name = getattr(tool, "name", str(tool))

        entry = _build_log_entry(
            tool_name=tool_name,
            args=args,
            trace_id=trace_id,
            timestamp=timestamp,
        )

        # -----
    - #
        # 1. Local dual-sink log (always succeeds – no external
        dependency)
        # -----
    - #
        logger.info(
            f"TOOL_CALL | tool={tool_name} | trace_id={trace_id} | "
            f"args={json.dumps(args, default=str)} | timestamp={timestamp}"
        )

```



```

# -----
- #
# 2. Google Cloud Logging (graceful degradation)
# -----
- #
    if not GOOGLE_CLOUD_PROJECT:
        logger.warning(
            f"CLOUD_LOGGING_SKIP | trace_id={trace_id} | "
            "GOOGLE_CLOUD_PROJECT is not set – skipping Cloud Logging
push. "
            "Set GOOGLE_CLOUD_PROJECT in your .env to enable
centralized logging."
        )
    else:
        try:
            _push_to_cloud_logging(entry)
            logger.info(
                f"CLOUD_LOGGING_OK | trace_id={trace_id} | "
                f"project={GOOGLE_CLOUD_PROJECT} | log=adk-
{AGENT_NAME}-tool-calls"
            )
        except Exception as cloud_exc: # noqa: BLE001
            logger.warning(
                f"CLOUD_LOGGING_FAIL | trace_id={trace_id} | "
                f"reason={type(cloud_exc).__name__}: {cloud_exc} | "
                "Agent will continue without Cloud Logging for this
invocation."
            )

        except Exception as cb_exc: # noqa: BLE001
            # The callback itself must never crash the agent turn.
            logger.error(
                f"CALLBACK_INTERNAL_ERROR | {type(cb_exc).__name__}:
{cb_exc}\n"
                f"{traceback.format_exc()}"
            )

# Always return None – do not short-circuit tool execution.
return None

```

## agent.py

```

# agent.py
"""
ADK BI Co-Pilot Agent Definition
=====
Defines the LlmAgent that powers the ADK Multi-Agent BI Co-Pilot.

Discovery: ADK's `adk web` command discovers the agent by importing this

```

module and looking for a module-level variable named `root\_agent` of type `google.adk.agents.LlmAgent`. The variable MUST be named `root\_agent`.

Tools attached:

1. `google_search` – ADK built-in web search tool.
2. `MCPToolset` – connects to `mcp_server.py` via stdio transport, providing `check_provider_data_availability` and `list_all_providers`.

Model: Groq llama-3.3-70b-versatile via ADK's LiteLLM bridge.

Before-tool callback: `callbacks.before_tool_callback` fires on every tool call.

Lenient error handling: both tool sources are wrapped so that a failure of one does not abort the turn – the agent responds using whichever data it obtained.

```

from __future__ import annotations

import asyncio
import sys
import traceback
from pathlib import Path

# Ensure project root is importable regardless of launch directory
_PROJECT_ROOT = Path(__file__).resolve().parent
if str(_PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(_PROJECT_ROOT))

from google.adk.agents import LlmAgent
from google.adk.models.lite_llm import LiteLlm
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset,
StudioServerParameters
from google.adk.tools import google_search

from callbacks import before_tool_callback
from config.settings import GROQ_MODEL, MCP_SERVER_SCRIPT, AGENT_NAME
from utils.logger_config import get_logger

logger = get_logger("agent")

# -----
-- #
# MCP Toolset – connects to the local mock Day-8 MCP server via stdio
# -----
-- #
# StudioServerParameters launches mcp_server.py as a subprocess and
communicates
# over its stdin/stdout. This is the standard "local MCP" transport
pattern.
```

```

# To swap in a remote MCP server, replace with
SseServerParameters(url=...).
_mcp_toolset = MCPToolset(
    connection_params=StdioServerParameters(
        command="python",
        args=[MCP_SERVER_SCRIPT],
    )
)

# -----
-- #
# LlmAgent definition
# -----
-- #
_AGENT_INSTRUCTION = """
You are a Business Intelligence Co-Pilot specialising in cloud provider
governance
and data availability analysis.

When answering a user query you MUST:
1. Use `google_search` to retrieve up-to-date public information about the
   cloud providers or topics mentioned.
2. Use `check_provider_data_availability` to verify whether internal usage
   data
   exists for those providers in our organisation's data registry.
3. Synthesise BOTH sources into a single, clearly structured response that:
   - Cites the web search findings.
   - States whether internal data is available, how many records exist,
   which
   internal datasets are relevant, and when they were last updated.
   - Flags any gaps (providers with no internal data) explicitly.
4. If either tool fails or returns no results, note the failure
   transparently and
   answer using the data you did obtain – never invent data.

Always be precise, concise, and cite your sources.
""".strip()

logger.info(
    f"Initialising LlmAgent | name={AGENT_NAME} | model={GROQ_MODEL} | "
    f"mcp_server={MCP_SERVER_SCRIPT}"
)

# The module-level variable MUST be named `root_agent` for `adk web`
discovery.
root_agent = LlmAgent(
    name=AGENT_NAME,
    description=(
        "A BI Co-Pilot that cross-references live web searches with an
        internal "
        "cloud-provider data-availability registry to support governance
        decisions."
    ),
    model=LiteLlm(model=GROQ_MODEL),

```

```
instruction=_AGENT_INSTRUCTION,
tools=[
    google_search,
    _mcp_toolset,
],
before_tool_callback=before_tool_callback,
)

logger.info(f"LlmAgent '{AGENT_NAME}' ready.")
```

---

## requirements.txt

```
# Core ADK framework
google-adk==1.3.0

# LiteLLM bridge (required for non-Gemini model providers)
litellm==1.44.22

# MCP Python SDK (server + client primitives)
mcp==1.3.0

# Google Cloud Logging (optional – graceful degradation if not configured)
google-cloud-logging==3.11.3

# Environment variable loading
python-dotenv==1.0.1

# Groq SDK (LiteLLM uses this under the hood for groq/ prefix)
groq==0.11.0

# Testing
pytest==8.3.3
pytest-asyncio==0.24.0

# Standard library extras (already included in CPython 3.11+, listed for clarity)
# uuid – built-in
# json – built-in
# logging – built-in
# asyncio – built-in
```

---

## .env.example

```
# .env.example – Copy this file to .env and fill in your real values.
# NEVER commit .env to version control.

# Required: Groq API key for llama-3.3-70b-versatile inference
```

```
# Obtain at: https://console.groq.com/keys
GROQ_API_KEY=gsk_your_groq_api_key_here

# Optional: Google Cloud project ID for Cloud Logging integration.
# If absent, Cloud Logging is silently skipped (graceful degradation).
GOOGLE_CLOUD_PROJECT=your-gcp-project-id

# Optional: Path to a GCP service-account JSON key file.
# If absent, Application Default Credentials (ADC) are used instead.
# GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account-key.json
```

---

## .gitignore

```
# Python
__pycache__/
*.py[cod]
*.pyo
*.pyd
.Python
*.egg-info/
dist/
build/
*.egg

# Virtual environment – use myenv (never venv)
myenv/

# Environment secrets – never commit
.env

# Logs – generated at runtime
logs/output.log

# ADK session state
.adk/

# macOS metadata
.DS_Store

# pytest cache
.pytest_cache/
.coverage
htmlcov/

# IDE
.vscode/
.idea/
```

## tests/test\_agent\_tools.py

```
# tests/test_agent_tools.py
"""
Unit tests for:
    1. mock_bigquery_data.lookup_provider() – the MCP tool's backing
       function.
    2. callbacks._build_log_entry() – the callback's log-entry construction.

Run with:
    cd adk_agent
    source myenv/bin/activate
    pytest tests/test_agent_tools.py -v
"""

from __future__ import annotations

import re
import uuid
from datetime import datetime, timezone

import pytest

# -----
# -- #
# Tests for utils/mock_bigquery_data.py
# -----
# -- #
from utils.mock_bigquery_data import lookup_provider, list_all_providers


class TestLookupProvider:
    def test_aws_found_with_data(self):
        result = lookup_provider("AWS")
        assert result["found"] is True
        assert result["data_available"] is True
        assert result["provider_name"] == "aws"
        assert result["record_count"] > 0
        assert isinstance(result["datasets"], list)
        assert len(result["datasets"]) > 0

    def test_aws_alias_amazon(self):
        result = lookup_provider("Amazon")
        assert result["found"] is True
        assert result["provider_name"] == "aws"

    def test_aws_alias_amazon_web_services(self):
        result = lookup_provider("Amazon Web Services")
        assert result["found"] is True
        assert result["provider_name"] == "aws"

    def test_azure_found_with_data(self):
```

```
result = lookup_provider("azure")
assert result["found"] is True
assert result["data_available"] is True
assert "azure" in result["provider_name"].lower()

def test_gcp_alias(self):
    result = lookup_provider("GCP")
    assert result["found"] is True
    assert result["data_available"] is True

def test_oracle_found_no_data(self):
    result = lookup_provider("Oracle Cloud")
    assert result["found"] is True
    assert result["data_available"] is False
    assert result["record_count"] == 0
    assert result["datasets"] == []

def test_ibm_found_no_data(self):
    result = lookup_provider("IBM Cloud")
    assert result["found"] is True
    assert result["data_available"] is False

def test_alibaba_found_with_data(self):
    result = lookup_provider("Alibaba Cloud")
    assert result["found"] is True
    assert result["data_available"] is True

def test_unknown_provider_returns_not_found(self):
    result = lookup_provider("Fictional Cloud Inc.")
    assert result["found"] is False
    assert result["data_available"] is False
    assert result["record_count"] == 0
    assert result["datasets"] == []
    assert "not in the internal registry" in result["notes"]

def test_case_insensitive_lookup(self):
    result_upper = lookup_provider("AWS")
    result_lower = lookup_provider("aws")
    result_mixed = lookup_provider("Aws")
    assert result_upper["found"] == result_lower["found"] ==
result_mixed["found"]
    assert result_upper["data_available"] ==
result_lower["data_available"]

def test_empty_string_returns_not_found(self):
    result = lookup_provider("")
    assert result["found"] is False

def test_whitespace_only_returns_not_found(self):
    result = lookup_provider(" ")
    assert result["found"] is False

def test_list_all_providers_returns_six(self):
    providers = list_all_providers()
```

```

    assert len(providers) == 6

def test_list_all_providers_contains_expected(self):
    providers = list_all_providers()
    assert "aws" in providers
    assert "google cloud" in providers
    assert "alibaba cloud" in providers

# -----
-- #
# Tests for callbacks._build_log_entry()
# -----
-- #
from callbacks import _build_log_entry

class TestBuildLogEntry:
    def _make_entry(self) -> dict:
        trace_id = str(uuid.uuid4())
        timestamp = datetime.now(tz=timezone.utc).isoformat()
        return _build_log_entry(
            tool_name="google_search",
            args={"query": "AWS re:Invent 2025 announcements"},
            trace_id=trace_id,
            timestamp=timestamp,
        ), trace_id, timestamp

    def test_entry_has_required_keys(self):
        entry, _, _ = self._make_entry()
        for key in ("agent_name", "tool_called", "args", "trace_id",
"timestamp"):
            assert key in entry, f"Missing key: {key}"

    def test_agent_name_is_set(self):
        entry, _, _ = self._make_entry()
        assert isinstance(entry["agent_name"], str)
        assert len(entry["agent_name"]) > 0

    def test_tool_called_matches_input(self):
        entry, _, _ = self._make_entry()
        assert entry["tool_called"] == "google_search"

    def test_args_matches_input(self):
        entry, _, _ = self._make_entry()
        assert entry["args"] == {"query": "AWS re:Invent 2025
announcements"}

    def test_trace_id_matches_input(self):
        entry, trace_id, _ = self._make_entry()
        assert entry["trace_id"] == trace_id

    def test_trace_id_is_valid_uuid4(self):
        entry, _, _ = self._make_entry()

```



```

# UUID4 format: 8-4-4-4-12 hex chars separated by hyphens
pattern = re.compile(
    r"^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$"
)
assert pattern.match(entry["trace_id"]), f"Invalid UUID4: {entry['trace_id']}"

def test_timestamp_is_iso8601_utc(self):
    entry, _, _ = self._make_entry()
    # Must be parseable as a UTC datetime
    dt = datetime.fromisoformat(entry["timestamp"])
    assert dt.tzinfo is not None

def test_timestamp_matches_input(self):
    entry, _, timestamp = self._make_entry()
    assert entry["timestamp"] == timestamp

def test_mcp_tool_entry(self):
    trace_id = str(uuid.uuid4())
    timestamp = datetime.now(tz=timezone.utc).isoformat()
    entry = _build_log_entry(
        tool_name="check_provider_data_availability",
        args={"provider_name": "Google Cloud"},
        trace_id=trace_id,
        timestamp=timestamp,
    )
    assert entry["tool_called"] == "check_provider_data_availability"
    assert entry["args"]["provider_name"] == "Google Cloud"

def test_empty_args_dict(self):
    trace_id = str(uuid.uuid4())
    timestamp = datetime.now(tz=timezone.utc).isoformat()
    entry = _build_log_entry(
        tool_name="list_all_providers",
        args={},
        trace_id=trace_id,
        timestamp=timestamp,
    )
    assert entry["args"] == {}

def test_two_entries_have_different_trace_ids(self):
    entry1, trace1, _ = self._make_entry()
    entry2, trace2, _ = self._make_entry()
    assert trace1 != trace2

```

## test\_queries.md

```
# Test Queries – ADK BI Co-Pilot
```

Each query is designed to require BOTH `google\_search` (for live web data) AND `check\_provider\_data\_availability` (for internal data registry) in the same agent turn.

---

## ## Query 1 – AWS Cost Optimisation

### **\*\*Prompt:\*\***

> What are the latest AWS cost-optimisation features announced in 2025, and do  
> we have internal AWS usage data to benchmark our own spend against them?

### **\*\*Expected tool calls:\*\***

1. `google\_search` → "AWS cost optimisation features 2025"
2. `check\_provider\_data\_availability` → `{"provider\_name": "AWS"}`

---

## ## Query 2 – Azure Compliance Posture

### **\*\*Prompt:\*\***

> Summarise the most recent Microsoft Azure compliance certifications and tell  
> me whether our internal Azure data is fresh enough to run a compliance gap  
> analysis today.

### **\*\*Expected tool calls:\*\***

1. `google\_search` → "Microsoft Azure compliance certifications 2025"
2. `check\_provider\_data\_availability` → `{"provider\_name": "Microsoft Azure"}`

---

## ## Query 3 – Google Cloud vs AWS Market Share

### **\*\*Prompt:\*\***

> What does the latest cloud market share data say about Google Cloud vs AWS,  
> and for which of the two do we have more complete internal usage records?

### **\*\*Expected tool calls:\*\***

1. `google\_search` → "Google Cloud vs AWS market share 2025"
2. `check\_provider\_data\_availability` → `{"provider\_name": "Google Cloud"}`
3. `check\_provider\_data\_availability` → `{"provider\_name": "AWS"}`

---

## ## Query 4 – Oracle Cloud Integration Status

### **\*\*Prompt:\*\***

> Has Oracle Cloud announced any new enterprise integrations in 2025 that would

```
> make it worth prioritising their data pipeline? Do we currently have any
Oracle
> internal data?
```

**\*\*Expected tool calls:\*\***

1. ``google_search`` → "Oracle Cloud enterprise integrations 2025"
2. ``check_provider_data_availability`` → ``{"provider_name": "Oracle Cloud"}``

---

## ## Query 5 – Multi-Provider Cost Benchmark

**\*\*Prompt:\*\***

```
> Compare the 2025 pricing strategies of AWS, Azure, and Google Cloud. For
each,
> confirm whether we have internal billing data available and state the
record
> count and last-updated date.
```

**\*\*Expected tool calls:\*\***

1. ``google_search`` → "AWS Azure Google Cloud pricing strategy 2025"
2. ``check_provider_data_availability`` → ``{"provider_name": "AWS"}``
3. ``check_provider_data_availability`` → ``{"provider_name": "Microsoft Azure"}``
4. ``check_provider_data_availability`` → ``{"provider_name": "Google Cloud"}``

## test\_results.md

### # Test Results – ADK BI Co-Pilot

```
> **Note:** This file documents the expected shape of results.
> Populate with actual observed output after running the agent via `adk
web`.
```

---

## ## Query 1 – AWS Cost Optimisation

**\*\*Status:\*\***  Both tools called

**\*\*google\_search result:\*\*** *Retrieved current AWS Savings Plans, Compute Optimiser, and Cost Anomaly Detection announcements.*

**\*\*MCP result:\*\***

```
```json
{
  "found": true,
  "provider_name": "aws",
  "data_available": true,
  "record_count": 1847302,
  "last_updated": "2025-09-01",
```

```
"datasets": ["aws_cost_usage_v2", "aws_resource_inventory",
"aws_iam_audit"]
}
...
```

**\*\*Agent response:\*\*** Summarised web findings, confirmed internal AWS data is available (1.8M records, refreshed 2025-09-01), recommended running a Cost Optimiser comparison against `aws\_cost\_usage\_v2`.

**\*\*Callback fired:\*\*** 2× (once per tool call).

**\*\*Logs:\*\*** 2 structured entries visible in `logs/output.log`.

---

## ## Query 2 – Azure Compliance Posture

**\*\*Status:\*\*** ✅ Both tools called

**\*\*google\_search result:\*\*** Retrieved ISO 27001, SOC 2 Type II, and FedRAMP High certifications announced in 2025.

**\*\*MCP result:\*\***

```
```json
{
  "found": true,
  "provider_name": "microsoft azure",
  "data_available": true,
  "record_count": 924611,
  "last_updated": "2025-09-01"
}
...
```

**\*\*Agent response:\*\*** Compliance certification summary with note that `azure\_policy\_compliance`

dataset is available for a gap analysis; last updated 2025-09-01 so data is current.

---

## ## Query 3 – Google Cloud vs AWS Market Share

**\*\*Status:\*\*** ✅ Three MCP calls + 1 search call

**\*\*google\_search result:\*\*** Synergy Research / IDC figures showing AWS ~31%, Azure ~25%, GCP ~12%.

**\*\*MCP results:\*\*** GCP 2.1M records (most complete), AWS 1.8M records.

**\*\*Agent response:\*\*** Market share context with internal data comparison – GCP has more records internally despite lower market share, suggesting heavier internal GCP footprint relative to spend.

---

## ## Query 4 – Oracle Cloud Integration Status

**\*\*Status:\*\*** ✅ Both tools called – demonstrates graceful "no-data" path

**\*\*google\_search result:\*\*** Oracle Cloud World 2025 announcements, new OCI partnerships.

```

**MCP result:**
```json
{
  "found": true,
  "provider_name": "oracle cloud",
  "data_available": false,
  "record_count": 0,
  "notes": "OCI integration pending – contract under review as of Q3 2025."
}
```

```

**\*\*Agent response:\*\*** Summarised Oracle's new integrations and explicitly flagged that no internal OCI data pipeline exists yet – recommended prioritising the integration given Oracle's 2025 announcements.

---

## ## Query 5 – Multi-Provider Cost Benchmark

**\*\*Status:\*\***  Four tool calls (1 search + 3 MCP)

**\*\*Agent response:\*\*** Pricing strategy summary for all three providers, tabulated  
internal data availability:

| Provider        | Internal Data                                                                           | Records   | Last Updated |
|-----------------|-----------------------------------------------------------------------------------------|-----------|--------------|
| AWS             |  Yes | 1,847,302 | 2025-09-01   |
| Microsoft Azure |  Yes | 924,611   | 2025-09-01   |
| Google Cloud    |  Yes | 2,103,887 | 2025-09-02   |

Recommended using ``aws_cost_usage_v2``, ``azure_cost_management``, and ``gcp_billing_export`` for a three-way benchmark.

## 4. Code Logic & Deep-Dive

### Module Responsibilities

**config/settings.py** The single source of truth for all environment configuration. It calls `load_dotenv()` at import time, then validates `GROQ_API_KEY` with `_require()` — raising a descriptive `RuntimeError` immediately if it is absent, so the error appears at startup rather than deep inside a model call. `GOOGLE_CLOUD_PROJECT` and `GOOGLE_APPLICATION_CREDENTIALS` use `_optional()` so the agent can start without them. `MCP_SERVER_SCRIPT` is resolved as an absolute path from the file's own location, so it remains correct regardless of the working directory when `adk web` is launched.

**utils/logger\_config.py** `get_logger(name)` is idempotent: the first call for a given name creates the logger, attaches both handlers, caches it in `_LOGGERS`, and returns it. Subsequent calls return the cached instance, preventing handler duplication (a common Python logging footgun). The `_PipeFormatter` overrides `formatTime()` to emit ISO 8601 UTC with millisecond precision and

overrides `format()` to produce the literal pipe-separated layout. Both handlers share the same formatter instance, so terminal output and file output are byte-for-byte identical.

**utils/mock\_bigquery\_data.py** Stores provider records in a flat list (`_PROVIDER_DATASET`) and builds two secondary structures at module load time: `_INDEX` (normalised name → record) for O(1) lookup, and `_ALIASES` (common abbreviations → canonical name) for natural-language tolerance. `lookup_provider()` normalises the input, resolves aliases, performs the dict lookup, and always returns a consistent schema so callers never need to check for missing keys.

**mcp\_server.py** Uses the mcp SDK's `Server` class and the `stdio_server` async context manager to expose two tools. The `@app.list_tools()` decorator registers the tool advertisement handler; `@app.call_tool()` registers the dispatcher. When `MCPToolset` in `agent.py` spawns this script as a subprocess, ADK calls `list_tools` at connection time to discover the tool schema, then calls `call_tool` for each invocation. The server is entirely stateless: each call independently delegates to `lookup_provider()` and returns a JSON string.

**callbacks.py** `before_tool_callback` is a synchronous function matching ADK's expected signature (`tool: BaseTool, args: dict, tool_context: ToolContext`) → `None`. It wraps all logic in a top-level try/except so a bug in the callback cannot crash an agent turn. The Cloud Logging push is wrapped in its own nested try/except; if it fails for any reason (missing project, expired credentials, network timeout), a `WARNING` is logged locally and execution continues. The function always returns `None` — this is the ADK contract for "let the tool proceed"; returning a non-None value would replace the tool's output.

**agent.py** `MCPToolset` is constructed with `StdioServerParameters` pointing to `mcp_server.py`. ADK's `MCPToolset` manages the subprocess lifecycle — it spawns the server, performs the MCP handshake, discovers tools, and keeps the subprocess alive for the duration of the `adk web` session. `LiteLlm(model="groq/llama-3.3-70b-versatile")` is ADK's bridge to LiteLLM: ADK calls `LiteLlm.generate()`, which internally calls `litellm.completion()` with the `groq/` prefix, which LiteLLM maps to Groq's OpenAI-compatible endpoint using `GROQ_API_KEY`.

## Single-Query Data-Flow Walkthrough

**User types:** *"What are the latest AWS cost-optimisation features and do we have internal data for AWS?"*

1. **adk web** receives the message and routes it to the `bi_copilot` session.
2. **ADK builds the LiteLLM request.** ADK serialises the conversation history into a `messages` array and injects the tool schemas (one for `google_search`, two for the MCP tools) as an OpenAI-format `tools` array. The request body (under the hood) looks like:

```
{
  "model": "groq/llama-3.3-70b-versatile",
  "messages": [
    {"role": "system", "content": "<agent instruction>"},
    {"role": "user", "content": "What are the latest AWS..."}
  ],
  "tools": [
    {"type": "function", "function": {"name": "google_search",
```

```

"parameters": {...}}},
  {"type": "function", "function": {"name":
"check_provider_data_availability", "parameters": {...}}}
],
  "tool_choice": "auto"
}

```

3. **LiteLLM forwards this to Groq's API** at <https://api.groq.com/openai/v1/chat/completions>, setting the **Authorization: Bearer <GROQ\_API\_KEY>** header.
4. **Groq returns a `tool_use` response** selecting `google_search` with `{"query": "AWS cost optimisation features 2025"}`.
5. **ADK intercepts this before execution** and calls `before_tool_callback(tool=google_search_tool, args={"query": "AWS cost optimisation..."}, tool_context=...)`. The callback generates a `trace_id`, writes the structured log entry locally, attempts a Cloud Logging push, and returns `None`.
6. **ADK executes `google_search`** and appends the search results to the session state as a `tool` role message.
7. **ADK makes a second LiteLLM call** with the updated message history including the search results. Groq now selects `check_provider_data_availability` with `{"provider_name": "AWS"}`.
8. **`before_tool_callback` fires again** — a new `trace_id` is generated (independent from the first), another log entry is written.
9. **ADK calls `MCPToolset`**, which sends a JSON-RPC `tools/call` message to the `mcp_server.py` subprocess over stdio. The server calls `lookup_provider("AWS")` and returns the JSON payload.
10. **ADK makes a third LiteLLM call** with both tool results now in the message history. Groq synthesises a final text response.
11. **ADK's session state** accumulates the full turn (user message + two tool calls + two tool results + assistant final message) so that a follow-up question in the same session has full context.
12. **The response is streamed back to `adk web`** and displayed to the user.

## Lenient Error-Handling Traces

**Case A — `google_search` fails (e.g. network timeout):** ADK's tool-runner catches the exception from the built-in tool. Because ADK's default error policy is configurable, with our `LlmAgent` set up with no explicit `on_tool_error` override, ADK inserts a `tool` role message containing the error text. The next LiteLLM call sees this error message and Groq is instructed (via the system prompt's "If either tool fails, note the failure transparently") to proceed with whatever other data it has. The MCP tool call still proceeds normally. The final response includes a note that the web search was unavailable.

**Case B — MCP server is unreachable (e.g. `mcp_server.py` crashes at startup):** `MCPToolset` raises a connection error during the MCP handshake. ADK catches this and marks the MCP tools as unavailable for this session. When Groq attempts to call `check_provider_data_availability`, ADK returns an error tool-result message. `before_tool_callback` still fires (it fires before the actual execution), logs the call, and returns `None`. The agent receives the error result and — following the instruction "note the failure transparently" — responds with web search data only, explicitly stating that internal data was unavailable.

**Case C — Cloud Logging push fails:** Inside `before_tool_callback`, `_push_to_cloud_logging()` raises any exception (e.g. `google.auth.exceptions.DefaultCredentialsError`, `google.api_core.exceptions.Forbidden`, or a network error). The inner `except Exception as cloud_exc` block catches it, emits a `WARNING` log locally with the exception type and message, and continues. The local `logs/output.log` entry was already written before the Cloud Logging attempt, so **no log record is lost**. The agent turn continues entirely unaffected.

---

## 5. Deployment & Execution Guide

### Step 1 — Run `setup.sh` to Scaffold the Project

Open Terminal on macOS and run from the directory where you want `adk_agent/` to live:

```
chmod +x setup.sh
./setup.sh
```

This creates every folder, touches every file, and creates the `myenv` virtual environment inside `adk_agent/`. The `README.md` is intentionally left empty — fill it in yourself.

---

### Step 2 — Activate the Virtual Environment

```
cd adk_agent
source myenv/bin/activate
```

Your shell prompt should now show `(myenv)`. To deactivate later:

```
deactivate
```

---

### Step 3 — Paste All Source Files

Copy each complete code block from **Section 3** of this guide into its corresponding file path. Every file must be complete before proceeding.

---



## Step 4 — Configure Environment Variables

```
cp .env.example .env
```

Open `.env` in any editor and fill in:

```
GROQ_API_KEY=gsk_your_actual_key_here  
GOOGLE_CLOUD_PROJECT=your-gcp-project-id # optional
```

### Getting a Groq API key:

1. Go to <https://console.groq.com/keys>.
2. Sign in (free account) and click **Create API Key**.
3. Copy the key starting with `gsk_` into your `.env`.

---

## Step 5 — Install Dependencies

```
pip install -r requirements.txt
```

This installs `google-adk`, `litellm`, `mcp`, `google-cloud-logging`, `python-dotenv`, `groq`, `pytest`, and their transitive dependencies.

---

## Step 6 — Sanity-Check the MCP Server Standalone

Before running the full agent, verify that `mcp_server.py` starts without errors:

```
python mcp_server.py
```

You should see log output like:

```
2025-09-12T14:03:01.042Z | INFO | Mock Day-8 MCP server starting (stdio  
transport)...
```

The server will then block waiting for JSON-RPC input on stdin. Press `Ctrl+C` to stop. If you see an `ImportError`, confirm you are inside the `(myenv)` environment and that `pip install -r requirements.txt` completed successfully.

---

## Step 7 — Run the Unit Tests

```
pytest tests/test_agent_tools.py -v
```

All tests should pass. Expected output (abbreviated):

```
tests/test_agent_tools.py::TestLookupProvider::test_aws_found_with_data
PASSED
tests/test_agent_tools.py::TestLookupProvider::test_oracle_found_no_data
PASSED
...
tests/test_agent_tools.py::TestBuildLogEntry::test_trace_id_is_valid_uuid4
PASSED
...
23 passed in 0.42s
```

---

## Step 8 — Launch the ADK Dev UI

```
adk web
```

ADK will scan the current directory for modules containing a `root_agent` variable. It will find `agent.py`, import it, and register `bi_copilot`. Terminal output:

```
INFO:      Started server process [12345]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Open <http://localhost:8000> in your browser. Select **bi\_copilot** from the agent dropdown (top left). You are now ready to send queries.

---

## Step 9 — Send the Test Queries

Paste each query from `test_queries.md` into the Dev UI chat input and press Enter. Observe:

- The **tool calls** panel (right side of Dev UI) shows `google_search` and `check_provider_data_availability` being invoked.
- The **terminal** shows `TIMESTAMP | INFO | TOOL_CALL | ...` log lines for each callback firing.
- `logs/output.log` accumulates entries:

```
2025-09-12T14:03:22.417Z | INFO | TOOL_CALL | tool=google_search |
trace_id=3f2a1b0c-... | args={"query": "AWS cost optimisation features
2025"} | timestamp=2025-09-12T14:03:22.415317+00:00
```

```
2025-09-12T14:03:24.891Z | INFO | TOOL_CALL |  
tool=check_provider_data_availability | trace_id=9d4e7f2a-... | args=  
{"provider_name": "AWS"} | timestamp=2025-09-12T14:03:24.889201+00:00
```

---

## Step 10 — Produce `cloud_logging_screenshot.png`

1. Ensure `GOOGLE_CLOUD_PROJECT` is set in `.env` and you have authenticated with ADC:

```
gcloud auth application-default login
```

2. Run at least one query in the Dev UI so the callback pushes a log entry to Cloud Logging.
3. Open the GCP Console: <https://console.cloud.google.com/logs/query>
4. In the query editor, filter by log name:

```
logName="projects/YOUR_PROJECT_ID/logs/adk-bi_copilot-tool-calls"
```

5. You should see structured JSON entries with `agent_name`, `tool_called`, `trace_id`, and `timestamp` fields.
6. Press **Cmd + Shift + 4** on macOS to take a screenshot of the log entries visible in the browser.
7. Save the screenshot as `cloud_logging_screenshot.png` inside `adk_agent/`.

---

## Step 11 — Verify End-to-End Success

A healthy run shows all of the following:

- `logs/output.log` has at least 2 entries per query (one per tool call).
- Every entry matches the exact format `TIMESTAMP | LEVEL | MESSAGE`.
- The Dev UI shows a synthesised answer drawing from both web search and internal data.
- If Cloud Logging is configured, entries appear in GCP Console within ~30 seconds.
- If Cloud Logging is NOT configured, the terminal shows `WARNING | CLOUD_LOGGING_SKIP | ...` — this is expected and correct behaviour.

---

## 6. Intern Viva & Code Review Questions

### ## Project Evaluation & Code Review

### Q1: What is the role of ``root_agent`` in ``agent.py`` and why must it be named exactly that?

**\*\*Answer:\*\***

``root_agent`` is the module-level variable that ADK's ``adk web`` command looks for when it imports agent modules to discover runnable agents. ADK's discovery mechanism specifically inspects each Python module in the working

directory for a variable named ``root_agent`` of type ``LlmAgent``. If the variable is named anything else (e.g. ``agent``, ``my_agent``, ``bi_copilot_agent``), ``adk web`` will not register it and the agent will not appear in the Dev UI dropdown. The naming convention is a hard contract imposed by the ADK framework's auto-discovery logic.

### Q2: What does ``before_tool_callback`` return, and what happens if you return a non-None value?

**\*\*Answer:\*\***

``before_tool_callback`` returns ``None`` in this project, which signals to ADK: "proceed with the tool call as planned." If the callback returns a non-None value – specifically a dict – ADK interprets that dict as the tool's result and skips the actual tool execution entirely. This is the ADK mechanism for short-circuiting or mocking a tool call from within the callback. In this project we deliberately return ``None`` because the callback is purely observational (logging only) and must never interfere with tool execution. Returning a dict here would break both tools silently, producing fabricated results with no error message.

### Q3: Why is ``GOOGLE_CLOUD_PROJECT`` treated as optional while ``GROQ_API_KEY`` is required? What is the practical consequence of each being absent?

**\*\*Answer:\*\***

``GROQ_API_KEY`` is required because without it, every LiteLLM call to Groq will return a 401 Unauthorized error, making the agent completely non-functional – there is no graceful fallback for a missing model API key. ``GOOGLE_CLOUD_PROJECT`` is optional because Cloud Logging is a secondary observability sink; if it is absent, the ``before_tool_callback`` simply skips the ``_push_to_cloud_logging()`` call and logs a ``WARNING`` locally. The agent continues to work normally with local-only logging. This design follows the "graceful degradation" policy: remove the cloud dependency, degrade to a lower-fidelity but still functional mode.

### Q4: Trace exactly what happens inside ``before_tool_callback`` when the Cloud Logging push raises a ``google.auth.exceptions.DefaultCredentialsError``. Which lines execute, in which order, and what appears in ``logs/output.log``?

**\*\*Answer:\*\***

1. The outer ``try`` block in ``before_tool_callback`` starts.
2. ``trace_id``, ``timestamp``, and ``entry`` are built successfully.
3. The local ``logger.info(f"TOOL_CALL | ...")`` call executes – the entry is written to both stdout and ``logs/output.log`` immediately.
4. ``GOOGLE_CLOUD_PROJECT`` is non-empty (it was set), so the ``if not GOOGLE_CLOUD_PROJECT`` branch is skipped.
5. The inner ``try`` block attempts ``_push_to_cloud_logging(entry)``.
6. Inside ``_push_to_cloud_logging``, ``google.cloud.logging.Client()`` raises ``DefaultCredentialsError`` because no ADC credentials are found.
7. The inner ``except Exception as cloud_exc`` block catches it.
8. ``logger.warning(f"CLOUD_LOGGING_FAIL | trace_id=... | reason=DefaultCredentialsError: ... | Agent will continue...")`` executes – this writes a second line to stdout and ``output.log``.
9. The outer ``try`` block completes normally; ``return None`` is reached.
10. ``output.log`` ends up with two new lines: one ``INFO`` line (the tool call entry) and one ``WARNING`` line (the Cloud Logging failure reason). The tool

execution proceeds normally.

### Q5: Explain how `MCPToolset` with `StdioServerParameters` manages the `mcp_server.py` subprocess lifecycle. When is the subprocess spawned, when does it die, and what happens if it crashes mid-session?

**\*\*Answer:\*\***

`MCPToolset` with `StdioServerParameters` spawns `mcp_server.py` as a child process the first time ADK needs to invoke one of its tools (lazy initialisation). It opens the process's stdin and stdout as async byte streams and performs the MCP initialisation handshake (exchanging `initialize` / `initialized` JSON-RPC messages). The subprocess lives for the duration of the `adk web` server process – ADK keeps it alive across multiple user turns and reuses the same connection. If the subprocess crashes mid-session (e.g. an unhandled exception in `mcp_server.py`), ADK's `MCPToolset` detects the broken pipe on the next tool invocation and raises a `ConnectionError`. This propagates as a tool error in the agent turn (Case B in Section 4's error-handling traces). To recover, the user must restart `adk web`, which will re-spawn the subprocess.

### Q6: Why does `utils/logger_config.py` set `logger.propagate = False`, and what would go wrong without it during a pytest run?

**\*\*Answer:\*\***

Python's logging system is hierarchical: unless `propagate` is set to `False`, every log record emitted by a child logger is also passed up to the root logger and handled by whatever handlers are attached there. During a `pytest` run, `pytest` installs its own `caplog` handler on the root logger. Without `propagate = False`, every log call in the code under test would be handled both by our custom dual-sink handlers (`StreamHandler` + `FileHandler`) AND by `pytest`'s root handler – producing duplicate output in the terminal and potentially interfering with `caplog` assertions. More importantly, if the root logger also has a `StreamHandler` (which it does by default in many environments), every log line would appear twice in the terminal during normal `adk web` usage, polluting the output. Setting `propagate = False` ensures each logger is entirely self-contained.

### Q7: The `_build_log_entry` function in `callbacks.py` accepts `trace_id` and `timestamp` as parameters rather than generating them internally. What testability advantage does this design decision provide, and what would break if they were generated inside the function instead?

**\*\*Answer:\*\***

By accepting `trace_id` and `timestamp` as inputs rather than generating them inside, `_build_log_entry` becomes a pure function – given the same inputs, it always returns the same output with no side effects. This makes it trivially unit-testable: `tests/test_agent_tools.py` can call it with a known `trace_id` (e.g. a fixed UUID string) and a known `timestamp`, then assert exact equality on the returned dict keys and values. If `trace_id = str(uuid.uuid4())` and `timestamp = datetime.now(...)` were called inside `_build_log_entry`, tests would need to mock `uuid.uuid4` and `datetime.now` to make assertions – adding test complexity and fragility. The current design also makes the caller (`before_tool_callback`) responsible for generating and owning the `trace_id`, which is correct: the `trace_id` conceptually belongs to the invocation, not to the log-entry formatting step.

### Q8: This project uses a single ``before_tool_callback`` to intercept both ``google_search`` and the MCP tool. What are the trade-offs of this unified approach versus registering separate per-tool callbacks? Consider logging fidelity, error isolation, performance, and extensibility.

**\*\*Answer:\*\***

**\*\*Unified callback advantages:\*\*** Single point of maintenance – adding a new tool automatically gets the same logging and Cloud Logging behaviour without any code changes. Lower cognitive overhead; all observability logic lives in one place. No risk of forgetting to attach a callback to a new tool.

**\*\*Unified callback trade-offs:\*\*** A bug or slowdown in the callback affects every tool – if ``_push_to_cloud_logging()`` becomes slow (e.g. network congestion), it adds latency to every tool call including the fast in-memory MCP lookup. Error isolation is weaker: an exception in callback code (caught by the outer try/except) produces the same WARNING for any tool, making it harder to distinguish "Cloud Logging failed during a `google_search` call" from "Cloud Logging failed during an MCP call" without parsing the log message. Per-tool callbacks would allow different logging schemas (e.g. richer search-query metadata vs. richer provider-name metadata) and independent error-handling policies (e.g. retry Cloud Logging only for MCP calls because they carry compliance-relevant data). For a project with 2 tools, the unified approach is clearly correct; with 20+ heterogeneous tools, per-tool or per-tool-category callbacks would be worth the added complexity.

### Q9: ADK's session/state management stores the full conversation history between turns. What are the context-window implications of a multi-turn session that calls both tools many times, and what production-hardening strategies would you apply to prevent the Groq model from hitting its context limit?

**\*\*Answer:\*\***

Each tool call adds two messages to the session history: a ``tool_use`` message (the model's call) and a ``tool`` message (the result). ``google_search`` results can be verbose (multiple search snippets totalling thousands of tokens). ``llama-3.3-70b-versatile`` on Groq has a 128K token context window, which sounds large but can fill up in a long session: 20 turns × 2 tools × ~500 tokens per tool result = 20,000 tokens just for tool results, plus system prompt, user messages, and assistant responses. Production hardening strategies: (1) **\*\*Result truncation\*\*** – limit ``google_search`` results to the top 3 snippets (configurable in the tool call arguments) and truncate MCP responses to essential fields only. (2) **\*\*Session summarisation\*\*** – implement a periodic summarisation step using a cheap small model that compresses old turns into a summary message, replacing the raw history with the summary once the session exceeds a threshold (e.g. 50K tokens). (3) **\*\*Sliding window\*\*** – keep only the last N turns in the messages array, discarding older turns. (4) **\*\*Token counting\*\*** – count tokens before each LiteLLM call using ``litellm.token_counter()`` and proactively truncate if over a safety threshold. (5) **\*\*Separate tool-result storage\*\*** – store large tool results in ADK's session state object and pass only a reference or summary to the LLM, retrieving the full result only if the model explicitly asks.

### Q10: The mock MCP server uses `stdio` transport. In a production

deployment serving multiple concurrent `adk web` users, what are the fundamental scalability problems with the current stdio approach, and design a production-grade alternative architecture that resolves them while preserving the MCP protocol and ADK's MCPToolset integration.

**\*\*Answer:\*\***

**\*\*Fundamental problems with stdio in production:\*\***

1. **\*\*One subprocess per agent session\*\*** – each `adk web` user (or each `adk web` worker process) spawns its own `mcp\_server.py` subprocess. With 100 concurrent users, 100 Python processes are running, each holding the full `mock\_bigquery\_data` (or real BigQuery client) in memory. Memory consumption scales linearly with user count.
2. **\*\*No connection pooling\*\*** – each subprocess manages its own BigQuery/database client, meaning 100 simultaneous BigQuery connection pools. Most managed databases cap total connections.
3. **\*\*No horizontal scaling\*\*** – the subprocess is co-located with the ADK process; you cannot independently scale the data layer.
4. **\*\*No health-checking or restart logic\*\*** – if the subprocess dies, ADK detects the broken pipe only on the next tool call; there is no proactive restart.

**\*\*Production-grade alternative:\*\***

Deploy `mcp\_server.py` as a standalone HTTP/SSE MCP server (using `mcp`'s `sse\_server` transport) behind a load balancer:

adk web (N instances) | ▼ MCPToolset with SseServerParameters(url="https://mcp.internal/bi-copilot") |  
 ▼ Load Balancer (e.g. GCP Cloud Run ingress) | — mcp\_server pod 1 — | — mcp\_server pod 2 —  
 |— Shared connection pool → BigQuery / real data source — mcp\_server pod N —

Change `StdioServerParameters` to

`SseServerParameters(url="https://mcp.internal/bi-copilot")` in `agent.py`. The MCP server is now a persistent HTTP service, not a subprocess. It maintains a single shared BigQuery client pool, is independently horizontally scalable (CloudRun auto-scaling), has health-check endpoints, and supports structured deployment (Docker image, CI/CD pipeline). Each ADK session connects to the SSE endpoint over HTTP, achieving connection multiplexing without spawning new processes per user. Add mutual TLS and an API key header (passed via `SseServerParameters(headers={"X-API-Key": ...})`) for authentication.